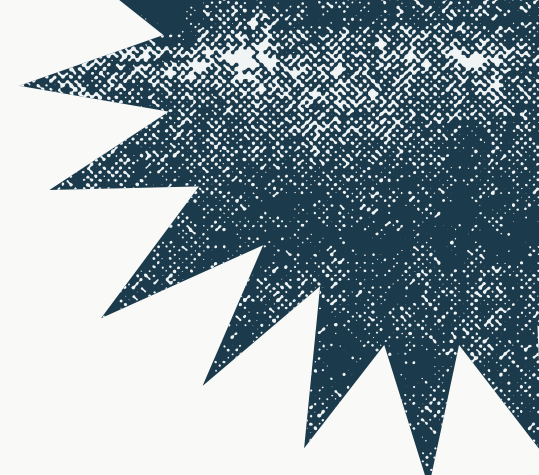




Prédiction de liens Gène–Maladie par Graph Mining

Module Graph Mining



Réaliser par :

- EL MAHDI MAJDI
- HAMMOU OUBINIZ

Encadrer par :

- PR. Amal Hjouji

Plan de la Présentation

01

Introduction

- Contexte biomédical
- Enjeux de la découverte gène-maladie
- Formulation du problème de prédiction de liens

02

Données et Graphe

- Base HPO
- Nettoyage des données,
- Construction du graphe biparti,
- Analyse structurelle

03

Préparation des Données

- Classification binaire
- Division Train/Test
- Génération d'exemples négatifs
- Validation qualité

04

Méthodes Baseline

- 6 méthodes heuristiques adaptées au graphe biparti
- Résultats
- Observations clés

05

Intelligence Artificielle

- Node2Vec et GAE
- comparaison avec les baselines
- analyse des résultats

06

Système et Conclusion

- Système de recommandation final
- Démonstrations concrètes
- Limites
- Perspectives

Contexte Biomédical

Le défi : Identifier les associations gène-maladie est un processus complexe, long (10-20 ans) et coûteux, ce qui peut nécessiter plusieurs années de recherche. Par conséquent, de nombreuses maladies restent sans gène identifié.

L'opportunité : Les bases de données biomédicales telles que HPO, OMIM et Orphanet regroupent aujourd'hui des milliers d'associations gène-maladie validées, constituant une source précieuse de connaissances pour la recherche.

Notre approche : Dans ce projet, nous modélisons les associations connues sous forme d'un **graphe biparti** et appliquons des méthodes de **prédiction de liens pour** mettre en évidence de nouvelles associations susceptibles d'exister.

6 000+

maladies mendéliennes
répertoriées

~60%

seulement ont un gène
causal identifié

15 940

associations gène-maladie dans HPO
utilisées pour entraîner nos modèles



Idée centrale : le Graph Mining permet d'exploiter la **structure topologique** du réseau gène-maladie pour prédire des liens manquants, accélérant ainsi la découverte de nouvelles associations biomédicales.

Problématique et Objectif

Graphe biparti : $G = (U \cup V, E)$ où U = gènes, V = maladies, E = associations connues

Tâche : Prédiction de liens

Notre objectif est d'estimer un score de probabilité pour chaque paire gène-maladie non connectée à partir du graphe partiel des associations connues.

Plus le score est élevé, plus l'association gène-maladie est probable.

Pipeline global (7 phases)

1. Données HPO

2. Nettoyage

3. Graphe

4. Train/Test

5. Baseline

6. IA

7. Reco

Sortie attendue

Recommandations top-K: pour chaque maladie, nous générons une liste des gènes les plus susceptibles d'y être associés, en sélectionnant les meilleurs candidats selon leur score de prédiction.

Évaluation : Les performances du modèle sont évaluées à l'aide de la métrique **AUC-ROC**, sur un ensemble de test masqué représentant 20 % des liens connus du graphe.

Objectif final : Atteindre une AUC supérieure à 0,6, indiquant une performance significativement meilleure qu'un modèle aléatoire.

Base de Données HPO

Human Phenotype Ontology (HPO) : ontologie standardisée de référence permettant de décrire les phénotypes des maladies humaines.

Elle fournit un vocabulaire contrôlé utilisé par les cliniciens et les chercheurs pour annoter de manière uniforme les caractéristiques des phénotypes des maladies génétiques.

Pourquoi HPO ?

- **Gratuite** et open-source (licence libre)
- **Fiable** : maintenue par le Jackson Laboratory
- **Académique** : référencée dans +2 000 publications
- **À jour** : mises à jour régulières avec la littérature
- **Structurée** : format tabulaire simple à exploiter

Fichier utilisé

genes_to_disease.txt

Fichier TSV associant gènes et maladies HPO.

Colonnes principales :

Colonne	Description
ncbi_gene_id	Identifiant NCBI du gène
gene_symbol	Symbole du gène (ex: TP53)
disease_id	Identifiant maladie (OMIM/ORPHA)

Nettoyage et Préparation

 **Problème identifié** : les identifiants gènes ont le format **NCBIGene:64170** – extraction du numéro uniquement requise.

Étapes de nettoyage

1

Extraction de l'ID numérique depuis NCBIGene:XXXX

2

Suppression des doublons (gène, maladie)

3

Filtrage des valeurs manquantes

4

Standardisation des identifiants maladie

Fichiers produits

Fichier	Contenu
gene_disease_edges.csv	Liste des arêtes (gene_id, disease_id)
gene_disease_full.csv	Données complètes avec symboles
genes_list.csv	Liste des gènes uniques
diseases_list.csv	Liste des maladies uniques

15 940

associations gène-maladie
validées et prêtes

	ncbi_gene_id	gene_symbol	association_type	disease_id	source
0	NCBIGene:64170	CARD9	MENDELIAN	OMIM:212050	ftp://ftp.ncbi.nlm.nih.gov/gene/DATA/mim2gene_...
1	NCBIGene:51256	TBC1D7	MENDELIAN	OMIM:248000	ftp://ftp.ncbi.nlm.nih.gov/gene/DATA/mim2gene_...
2	NCBIGene:28981	IFT81	MENDELIAN	OMIM:617895	ftp://ftp.ncbi.nlm.nih.gov/gene/DATA/mim2gene_...
3	NCBIGene:8216	LZTR1	MENDELIAN	OMIM:616564	ftp://ftp.ncbi.nlm.nih.gov/gene/DATA/mim2gene_...
4	NCBIGene:6505	SLC1A1	POLYGENIC	OMIM:615232	ftp://ftp.ncbi.nlm.nih.gov/gene/DATA/mim2gene_...
5	NCBIGene:5949	RBP3	MENDELIAN	OMIM:615233	ftp://ftp.ncbi.nlm.nih.gov/gene/DATA/mim2gene_...
6	NCBIGene:583	BBS2	MENDELIAN	OMIM:616562	ftp://ftp.ncbi.nlm.nih.gov/gene/DATA/mim2gene_...
7	NCBIGene:4750	NEK1	POLYGENIC	OMIM:617892	ftp://ftp.ncbi.nlm.nih.gov/gene/DATA/mim2gene_...
8	NCBIGene:51569	UFM1	MENDELIAN	OMIM:617899	ftp://ftp.ncbi.nlm.nih.gov/gene/DATA/mim2gene_...
9	NCBIGene:392255	GDF6	MENDELIAN	OMIM:617898	ftp://ftp.ncbi.nlm.nih.gov/gene/DATA/mim2gene_...



	gene_id	gene_symbol	disease_id	association_type
0	64170	CARD9	OMIM:212050	MENDELIAN
1	51256	TBC1D7	OMIM:248000	MENDELIAN
2	28981	IFT81	OMIM:617895	MENDELIAN
3	8216	LZTR1	OMIM:616564	MENDELIAN
4	6505	SLC1A1	OMIM:615232	POLYGENIC
5	5949	RBP3	OMIM:615233	MENDELIAN
6	583	BBS2	OMIM:616562	MENDELIAN
7	4750	NEK1	OMIM:617892	POLYGENIC
8	51569	UFM1	OMIM:617899	MENDELIAN
9	392255	GDF6	OMIM:617898	MENDELIAN

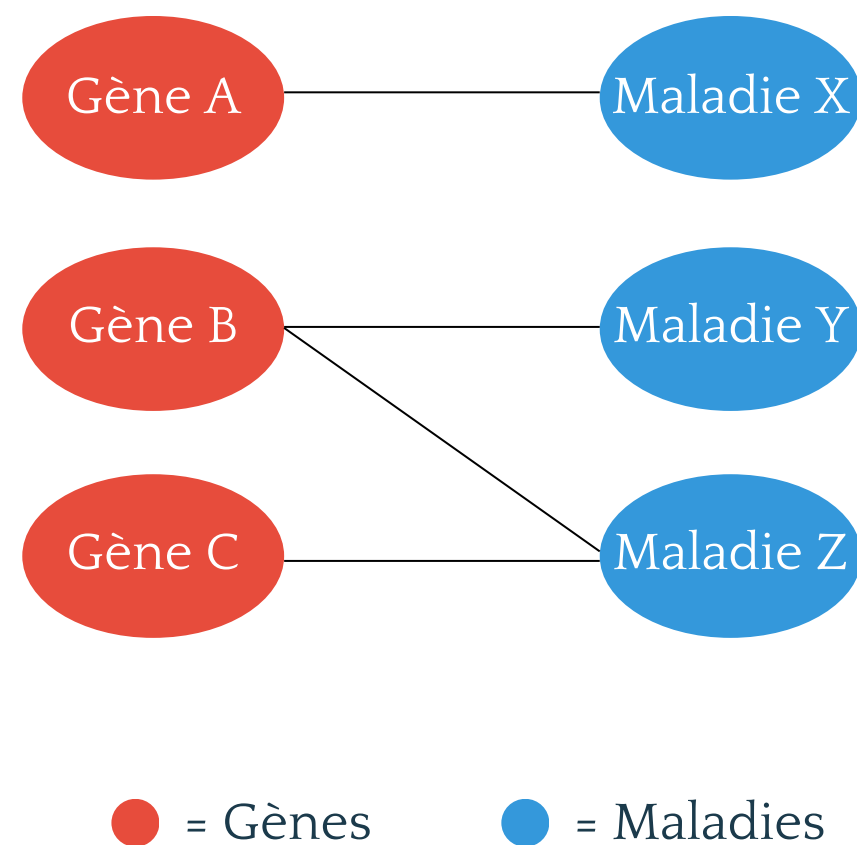
	gene_id	disease_id
0	64170	OMIM:212050
1	51256	OMIM:248000
2	28981	OMIM:617895
3	8216	OMIM:616564
4	6505	OMIM:615232
5	5949	OMIM:615233
6	583	OMIM:616562



Construction du Graphe Biparti

Définition : Un graphe biparti est un graphe composé de deux ensembles de nœuds disjoints U (gènes) et V (maladies), dans lequel les arêtes ne peuvent exister qu'entre les deux ensembles, et jamais entre deux nœuds appartenant au même ensemble.

Schéma du graphe



Implémentation NetworkX

1. Création du graphe : `nx.Graph()`
2. Ajout des nœuds gènes avec attribut `node_type='gene'`
3. Ajout des nœuds maladies avec `node_type='disease'`
4. Ajout des arêtes depuis le CSV nettoyé
5. Vérification : `nx.is_bipartite(G) = True`

Chemin de longueur 2

Deux gènes sont reliés par un chemin de longueur 2 s'ils partagent une maladie commune. C'est la base des méthodes de similarité : **Gène A → Maladie X → Gène B**

Analyse Structurale du Graphe

Statistiques clés



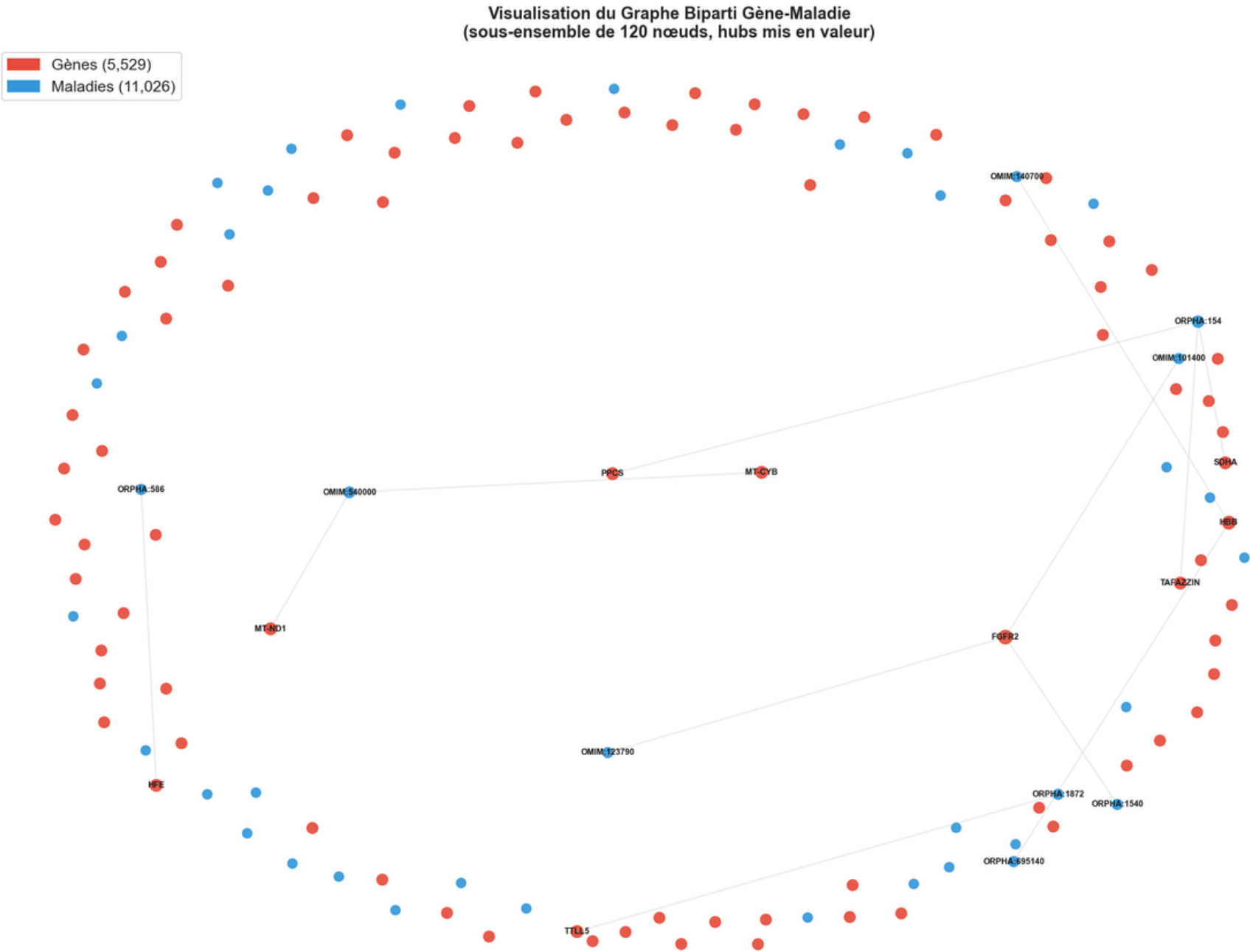
Distribution des degrés : loi de puissance typique – peu de hubs, beaucoup de nœuds faiblement connectés.

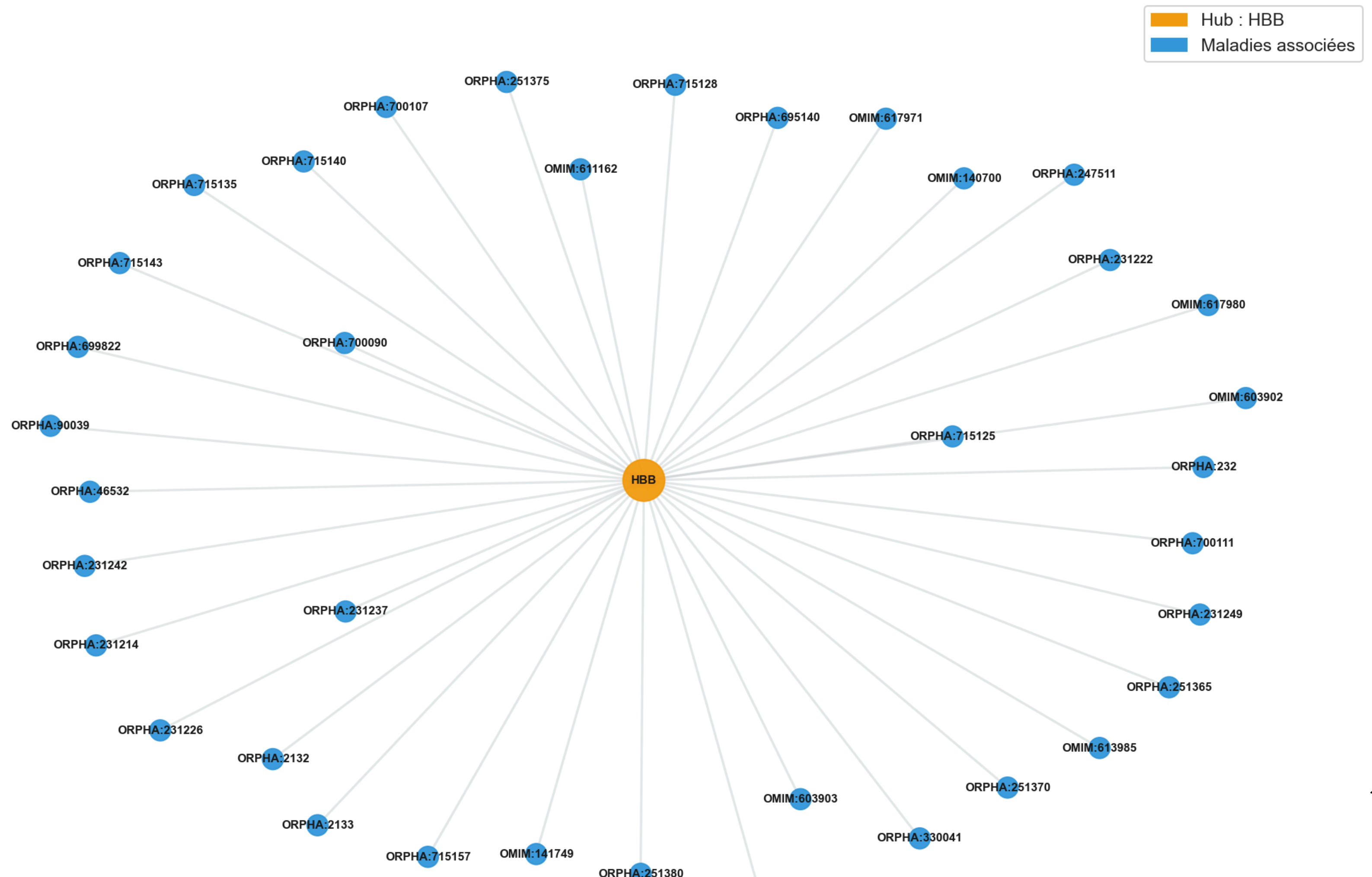
Hubs identifiés :

- TP53 – 89 maladies (suppresseur de tumeur)
- TTN – 72 maladies (protéine musculaire)

Nœuds GÈNES	:	5,529
Nœuds MALADIES	:	11,026
Total nœuds	:	16,555
Total arêtes	:	15,940

Nœuds gènes	:	5,529
Nœuds maladies	:	11,026
Arêtes (associations)	:	15,940
Densité	:	0.000261
Degré moyen (gènes)	:	2.88 maladies/gène
Degré moyen (maladies)	:	1.45 gènes/maladie





Problème de Classification Binaire

La prédiction de liens est transformée en un **problème de classification binaire supervisée**. Chaque paire gène-maladie reçoit un label :

Label = 1 Association réelle (positive)

Les paires (gène, maladie) reliées par une arête dans le graphe HPO correspondent à des associations déjà validées et confirmées scientifiquement.

Label = 0 Non-lien (négative)

Paires sans arête dans le graphe. Tirage aléatoire parmi les paires non connectées. Certaines pourraient être de vrais liens non découverts.

Schéma conceptuel

Existant dans HPO

- TP53 → Cancer du sein
- HBB → Drépanocytose
- CFTR → Mucoviscidose

Non-lien (aléatoire)

- TP53 → Alzheimer
- HBB → Diabète type 1
- CFTR → Parkinson

Modèle de classification apprend à distinguer les associations réelles des paires non liées.

Objectif d'apprentissage : Le modèle a pour objectif d'apprendre à distinguer les véritables associations, en exploitant les motifs structuraux du graphe, des paires aléatoires. Il doit ensuite généraliser ces apprentissages afin de prédire de nouvelles associations.

Division Train/Test et Exemples Négatifs

Règle 80/20

Séparation des données : Les arêtes positives sont divisées à 80 % pour l'entraînement et à 20 % pour le test.

Les exemples négatifs sont générés par tirage aléatoire parmi les paires de nœuds non connectés.

TRAIN (80%)

- 25 504 exemples
- 12 752 positifs (vrais liens)
- 12 752 négatifs (non-liens)

TEST (20%)

- 6 376 exemples
- 3 188 positifs (vrais liens)
- 3 188 négatifs (non-liens)

Ratio global

- 50 positifs - 50 négatifs
- Jeu équilibré
- Total : 31 880 paires

Méthode de génération des négatifs

Tirage aléatoire uniforme parmi toutes les paires (gène, maladie) **n'ayant pas d'arête** dans le graphe original. Le même nombre de négatifs que de positifs est généré pour chaque split.

⚠ **Attention** : certains "négatifs" pourraient être de vrais liens non encore découverts — c'est une limite inhérente à l'approche.

```

Total arêtes      : 15,940
Train (80%)       : 12,752
Test (20%)        : 3,188

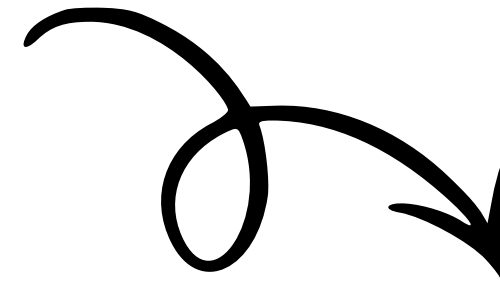
```

```

Génération terminée !
Tentatives effectuées : 15,944
Négatifs générés      : 15,940
Négatifs TRAIN        : 12,752
Négatifs TEST         : 3,188

Aperçu de quelques exemples négatifs train :
✗ G_51106 —X— D_ORPHA:276238
✗ G_779   —X— D_OMIM:261990
✗ G_10112 —X— D_ORPHA:140917
✗ G_57731 —X— D_OMIM:614653
✗ G_144245 —X— D_ORPHA:98954

```



```

Aperçu de quelques arêtes train positives :
✓ G_2694  → D_ORPHA:332
✓ G_64072 → D_ORPHA:90636
✓ G_6716  → D_ORPHA:1331
✓ G_859   → D_ORPHA:101016
✓ G_7248  → D_ORPHA:805
...

Aperçu de quelques arêtes TEST positives (cachées) :
✓ G_7287  → D_ORPHA:791
✓ G_653509 → D_OMIM:619611
✓ G_6496  → D_ORPHA:220386
✓ G_55384 → D_ORPHA:254531
✓ G_27063 → D_ORPHA:154

```

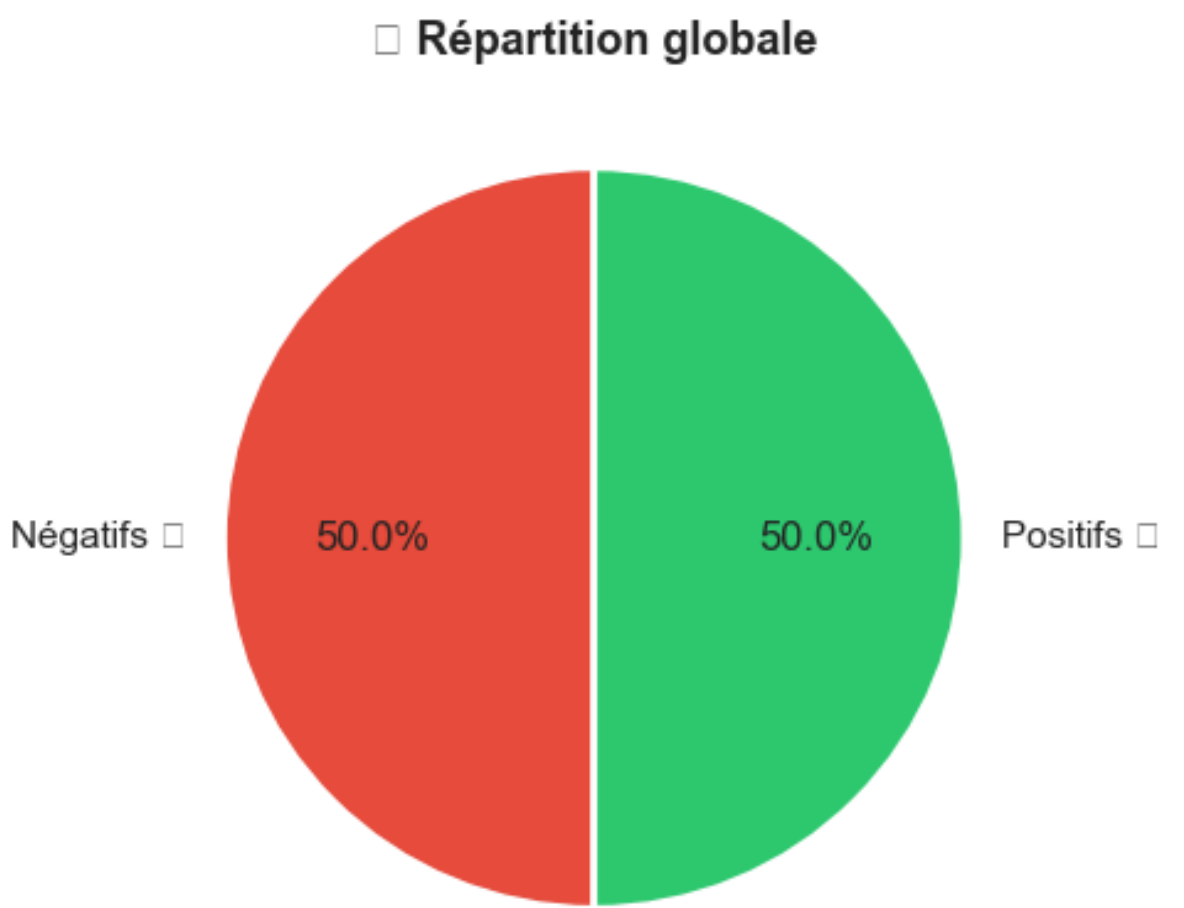
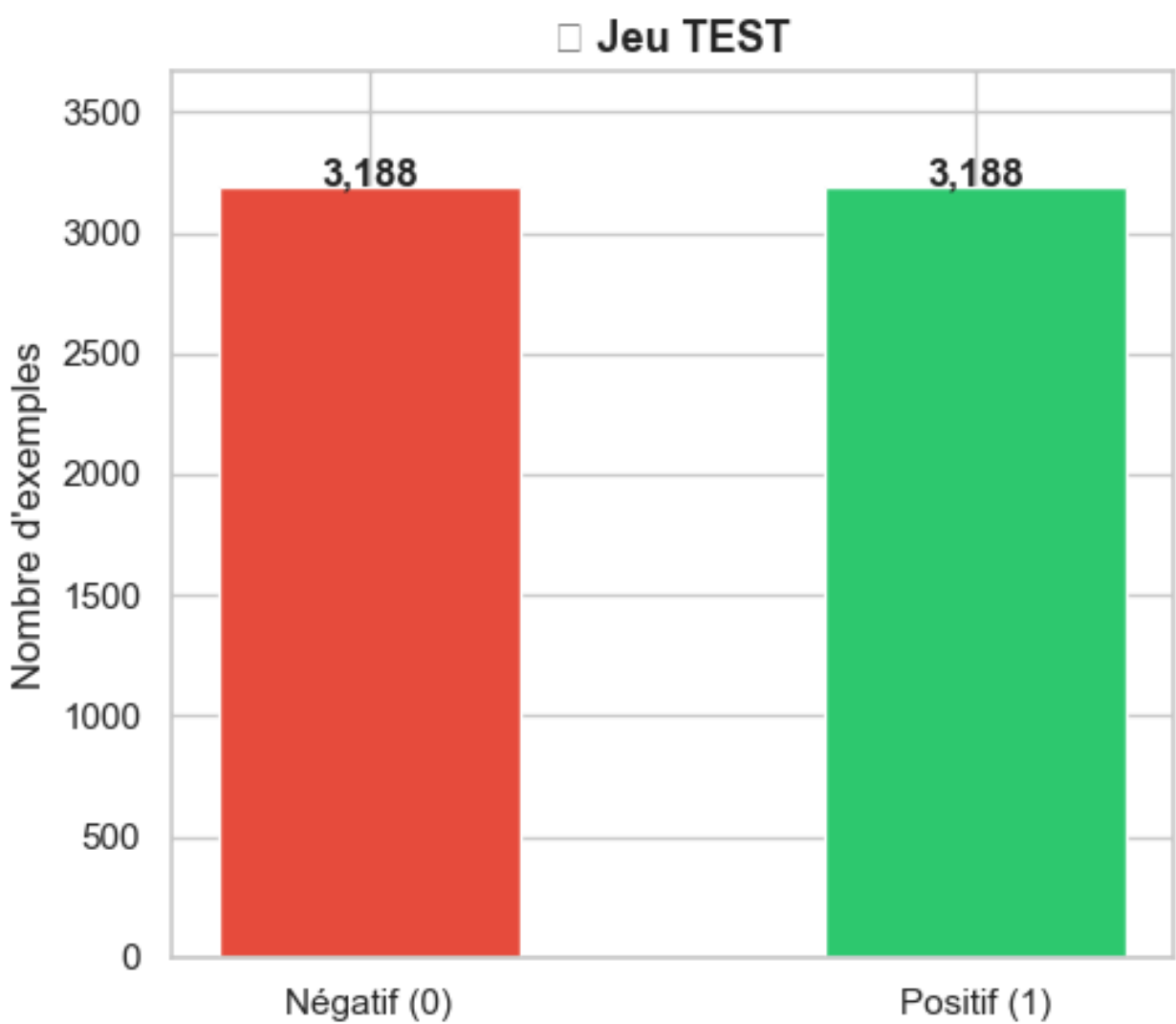
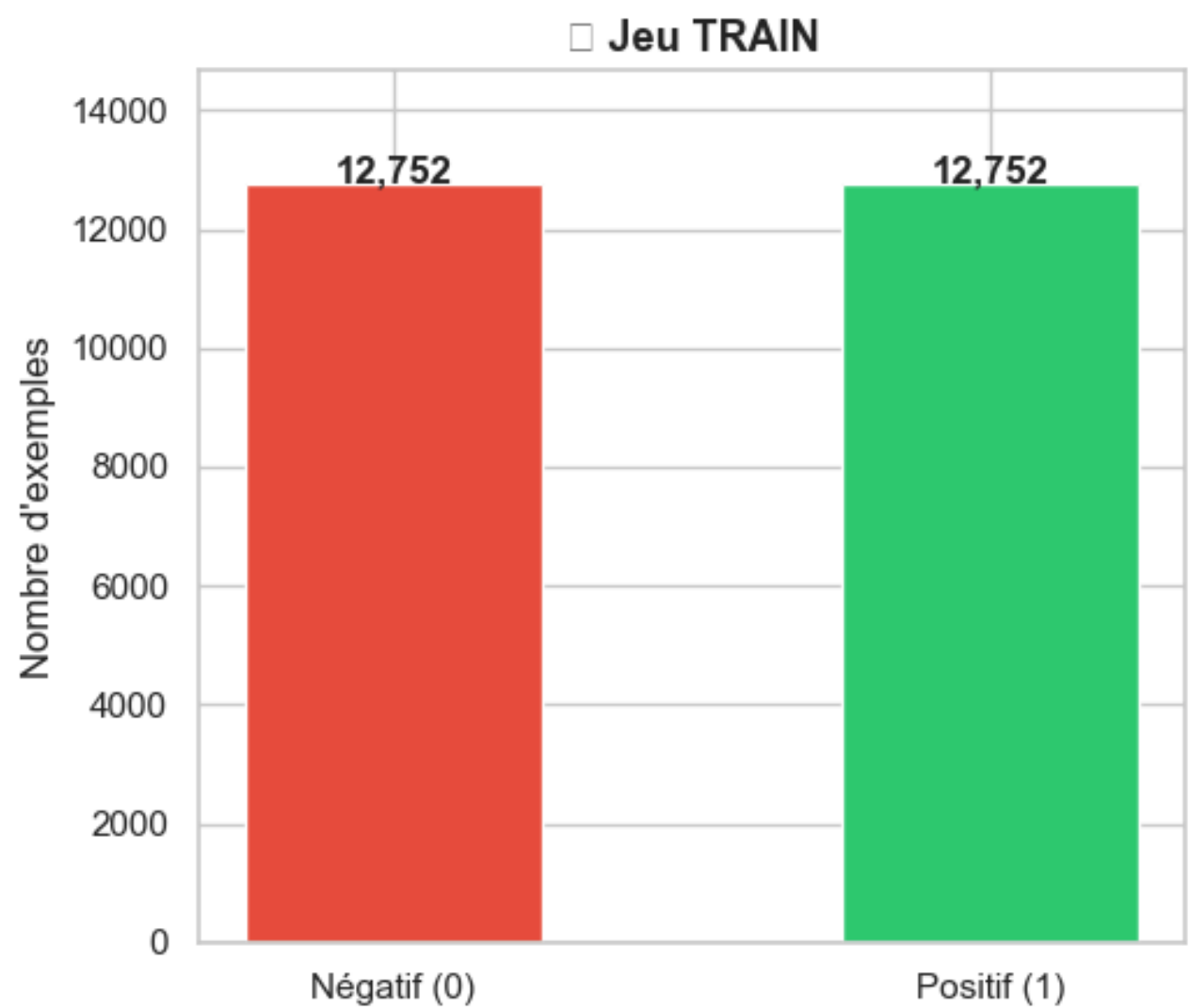
```

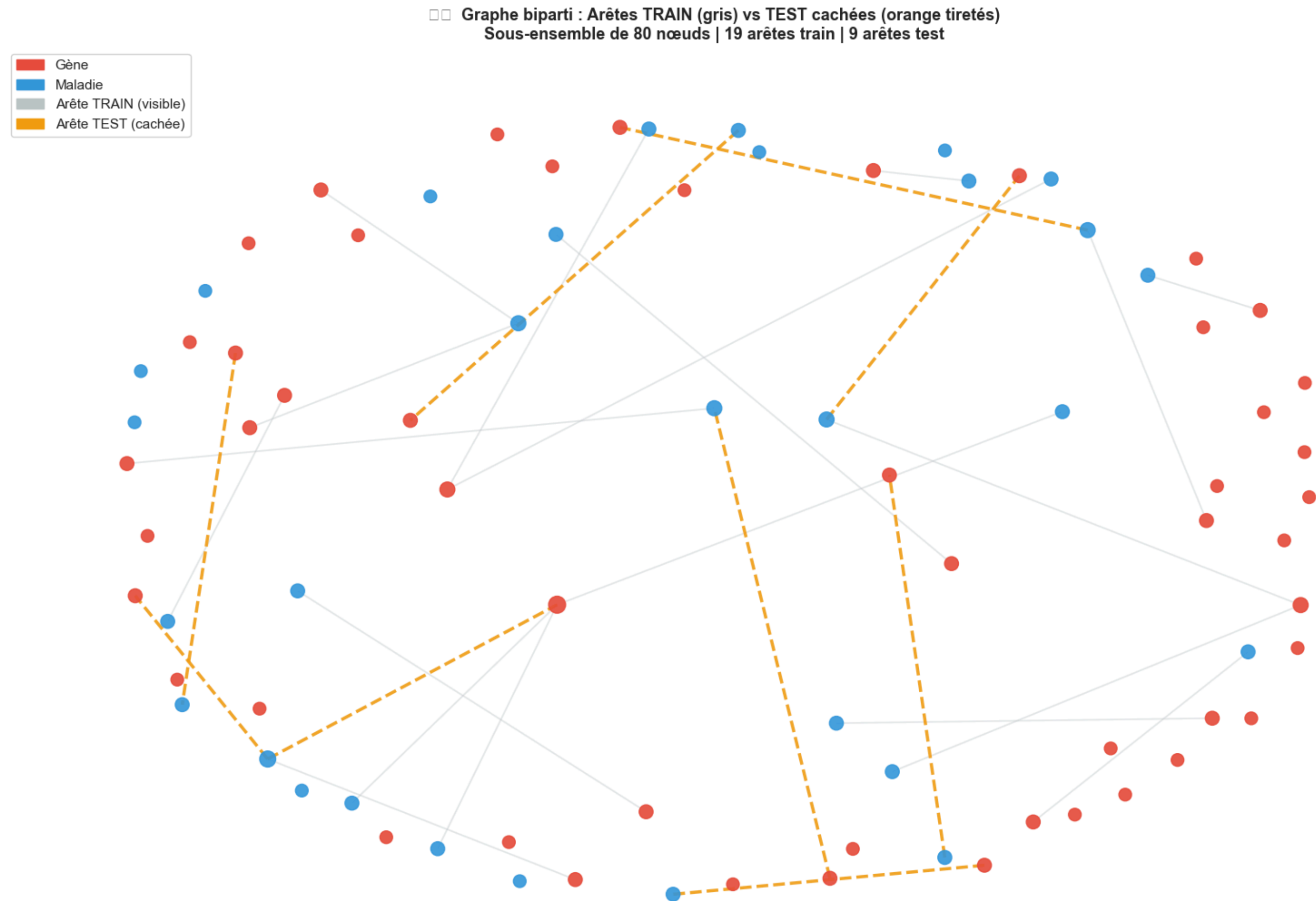
TRAIN total      : 25,504 exemples
  Positifs (1)   : 12,752
  Négatifs (0)   : 12,752
-----
TEST total       : 6,376 exemples
  Positifs (1)   : 3,188
  Négatifs (0)   : 3,188

```


	node1	node2	label
0	G_22827	D_ORPHA:404499	0
1	G_473	D_OMIM:616975	1
2	G_120892	D_ORPHA:2828	1
3	G_3630	D_OMIM:618858	1
4	G_57654	D_OMIM:621214	0
5	G_10568	D_OMIM:613686	0
6	G_90416	D_ORPHA:48818	0
7	G_2189	D_ORPHA:84	1
8	G_50511	D_OMIM:270960	1
9	G_6857	D_OMIM:614700	0

□ □ Équilibre des classes (positif / négatif)





Validation de la Qualité

5 tests automatiques ont été mis en place afin de garantir l'absence de **fuite de données (data leakage)** entre les ensembles **train** et **test** :

✓ **Test 1** : Pas de fuite train/test: aucune arête test n'apparaît dans le graph G_train

✓ **Test 2** : Tous les liens positifs train sont présents dans G_train

✓ **Test 3** : Aucun positif test n'apparaît dans G_train

✓ **Test 4** : Tous les négatifs sont absents du graphe original

✓ **Test 5** : Classes équilibrées (50% positifs / 50% négatifs)

✓ **Résultat** : 5/5 tests passés

Code de validation

```
def validate_split(G_train, pos_train, pos_test, neg_all, G_full):  
    assert not set(pos_test) & set(G_train.edges())      # Test 1  
    assert set(pos_train) <= set(G_train.edges())        # Test 2  
    assert all(not G_train.has_edge(u,v) for u,v in pos_test) # Test 3  
    assert all(not G_full.has_edge(u,v) for u,v in neg_all) # Test 4, 5
```

```
Test 1 - Pas de fuite train/test      : True (overlap=0)  
Test 2 - Positifs train dans G_train : True (12752/12752)  
Test 3 - Positifs test absents de G_train : True (3188/3188)  
Test 4 - Négatifs absents du graphe : True (1000/1000 vérifiés)  
Test 5 - Classes équilibrées          : True (ratio=0.500)
```

Principe des Méthodes Heuristiques

Pourquoi des baselines ?

Méthodes heuristiques = référence de base (baseline).

Un modèle d'IA doit les surpasser pour démontrer sa valeur ajoutée.

Idée générale

Score de similarité de voisinage : plus deux nœuds partagent de voisins communs, plus ils ont de chances d'être liés. Basé sur l'hypothèse que le graphe a une structure de **communautés**.

6 méthodes testées : Preferential Attachment, Common Neighbors, Jaccard, Adamic-Adar, Resource Allocation (toutes adaptées au biparti), Score Combiné.

Avantages

- Rapides (pas d'entraînement)
- Interprétables
- Sans hyperparamètres
- Fonctionnent sur CPU

Limites

- Vision locale uniquement
- 2 sauts maximum
- Ignore la structure globale
- Échouent sur graphes creux

Principe commun : utiliser les voisins du 2ème degré pour calculer la similarité entre un gène et une maladie

$$N^2(\text{gène}) = \{\text{gènes partageant une maladie avec le gène cible}\}$$

Problème Spécifique au Graphe Biparti

Problème : dans un graphe biparti, toujours

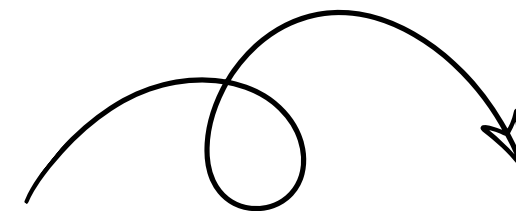
- **Dans un graphe biparti:** les voisins d'un gène sont uniquement des maladies, tandis que les voisins d'une maladie sont uniquement des gènes. Les deux ensembles étant disjoints par définition, ils ne partagent aucun voisin direct.
- Par conséquent, les mesures de similarité classiques telles que **Common Neighbors (CN)**, **Jaccard**, **Adamic-Adar (AA)** ou **Resource Allocation (RA)** retournent systématiquement un score nul lorsqu'elles sont appliquées directement au graphe biparti.

Formules classiques (échec)

$$CN(u, v) = |N(u) \cap N(v)| = 0$$

$$Jaccard(u, v) = \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|} = 0$$

$$AA(u, v) = \sum_{w \in N(u) \cap N(v)} \frac{1}{\log |N(w)|} = 0$$



Formules adaptées (solution)

$$CN_{2dp}(g, d) = |N^2(g) \cap N(d)|$$

$$N^2(g) = \{g' : \exists m, g \rightarrow m \rightarrow g'\}$$

Voisins du 2ème degré via les maladies

Solution adoptée : au lieu de chercher les voisins communs directs (impossible), on calcule les voisins du **2ème degré** : les gènes qui partagent au moins une maladie avec le gène cible. Puis on applique les formules classiques sur ce nouvel ensemble. Cette adaptation est **spécifique au graphe biparti** et a permis d'obtenir des scores non nuls pour la première fois.

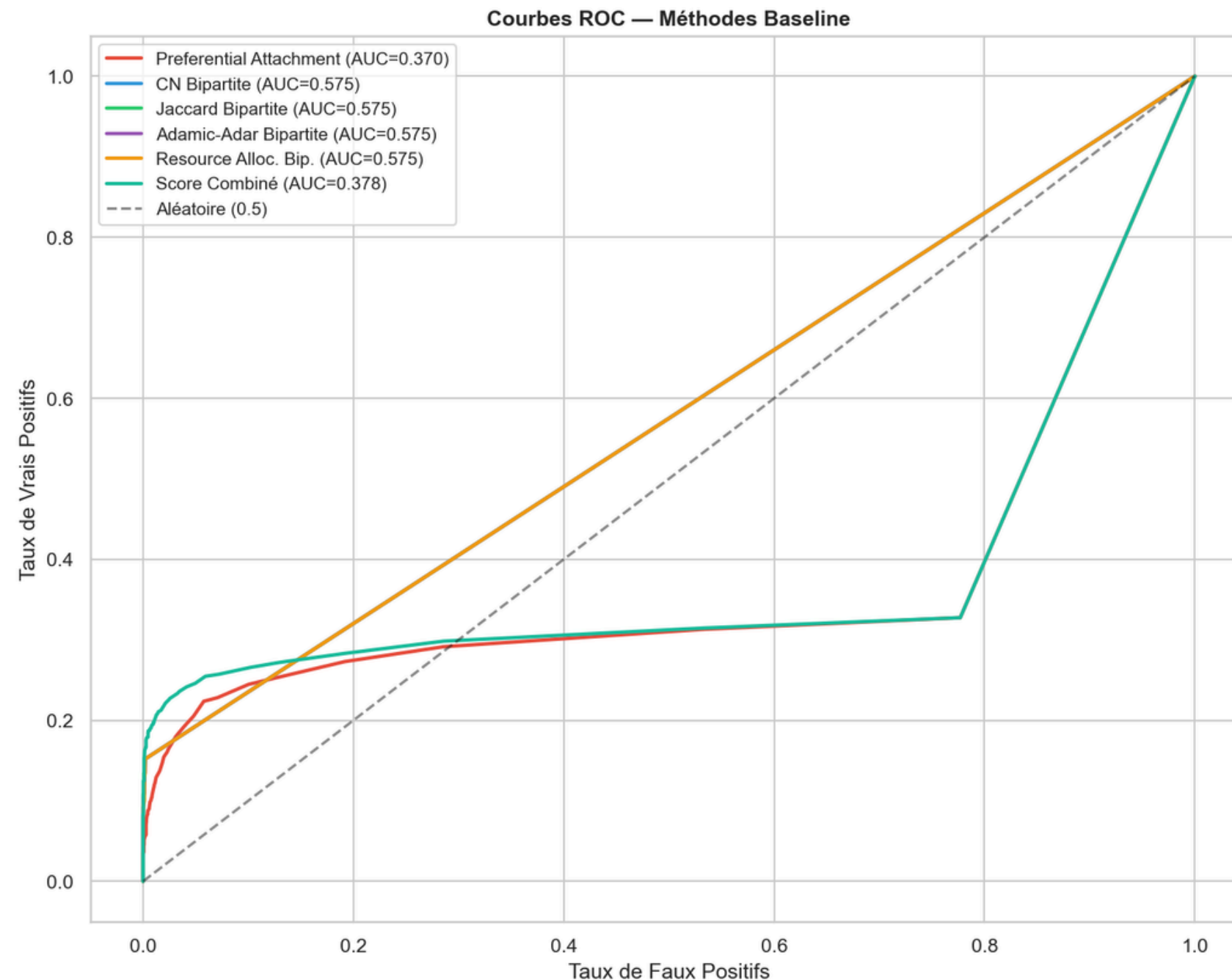
	node1	node2	label	score_PA	score_CN	score_JC	score_AA	score_RA	score_COMB
0	G_92002	D_OMIM:185800	0	1.0	0.0	0.0	0.0	0.0	0.000068
1	G_4204	D_ORPHA:777	1	184.0	0.0	0.0	0.0	0.0	0.012494
2	G_8788	D_ORPHA:169154	0	5.0	0.0	0.0	0.0	0.0	0.000340
3	G_1029	D_OMIM:621060	0	8.0	0.0	0.0	0.0	0.0	0.000543
4	G_93035	D_OMIM:618097	0	1.0	0.0	0.0	0.0	0.0	0.000068
5	G_26284	D_OMIM:617565	1	0.0	0.0	0.0	0.0	0.0	0.000000
6	G_4921	D_OMIM:618175	1	0.0	0.0	0.0	0.0	0.0	0.000000
7	G_3239	D_OMIM:113200	1	0.0	0.0	0.0	0.0	0.0	0.000000

Les 6 Méthodes Implémentées

Méthode	Description	Formule	AUC
Preferential Attachment	Produit des degrés : hypothèse 'rich get richer'	$PA(u, v) = \deg(u) \times \deg(v)$	0.3704
CN Bipartite	Nombre de voisins du 2ème degré communs	$ N^2(u) \cap N(v) $	0.5751
Jaccard Bipartite	CN normalisé par l'union des voisinages	$\frac{ N^2(u) \cap N(v) }{ N^2(u) \cup N(v) }$	0.5751 ★
Adamic-Adar Bip.	CN pondéré : voisins rares = plus informatifs	$\sum \frac{1}{\log N(w) }$	0.5751
Resource Alloc. Bip.	CN pondéré : pénalisation forte des hubs	$\sum \frac{1}{ N(w) }$	0.5750
Score Combiné	Combinaison PA normalisé + CN normalisé	$0.6 \times PA_{norm} + 0.4 \times CN_{norm}$	0.3784

Observation : Jaccard, CN, Adamic-Adar et Resource Allocation ont des AUC quasi-identiques (~0.575) car ils reposent sur le même ensemble de voisins du 2ème degré. Le Score Combiné et PA sous-performent significativement.

Résultats Baseline et 3 Observations Clés



3 observations clés

- 1** **AUC ~0.57** — signal faible mais supérieur au hasard (0.5).
Les heuristiques capturent une information locale limitée.
- 2** **P@K = 100%** est un artefact de petites composantes isolées (structures K2,2), pas un vrai signal généralisable.
- 3** **PA AUC = 0.37 < hasard** — les gènes rares ont statistiquement plus de nouveaux liens que les hubs.
Cohérent avec la génétique mendélienne.

Conclusion baseline : les heuristiques locales sont insuffisantes sur ce graphe biparti creux. L'AUC de 0.575 est une référence à battre — l'IA doit apporter un gain significatif pour justifier son usage.

Pourquoi l'IA est Nécessaire Ici

Vision locale (baselines)

Portée : 2 sauts maximum dans le graphe

Information : voisins directs et voisins du 2ème degré

Résultat : AUC = 0.575 (peu au-dessus du hasard)

Incapables de capturer les structures globales : communautés de gènes, patterns à longue distance, similarités sémantiques implicites.

VS

Vision globale (IA)

Portée : structure globale du graphe entier

Information : embeddings = représentations vectorielles

Résultat : AUC = 0.696 (+12 points)

Capables de détecter des patterns invisibles aux méthodes locales : communautés, proximités sémantiques, structures multi-sauts.

Deux approches IA testées

Node2Vec – marches aléatoires + Skip-gram

Approche locale par échantillonnage de chemins.

Analogie Word2Vec sur graphes.

GAE – factorisation SVD de la matrice d'adjacence

Approche globale par décomposition matricielle. Capture la structure complète.

Node2Vec – Principe et Paramètres

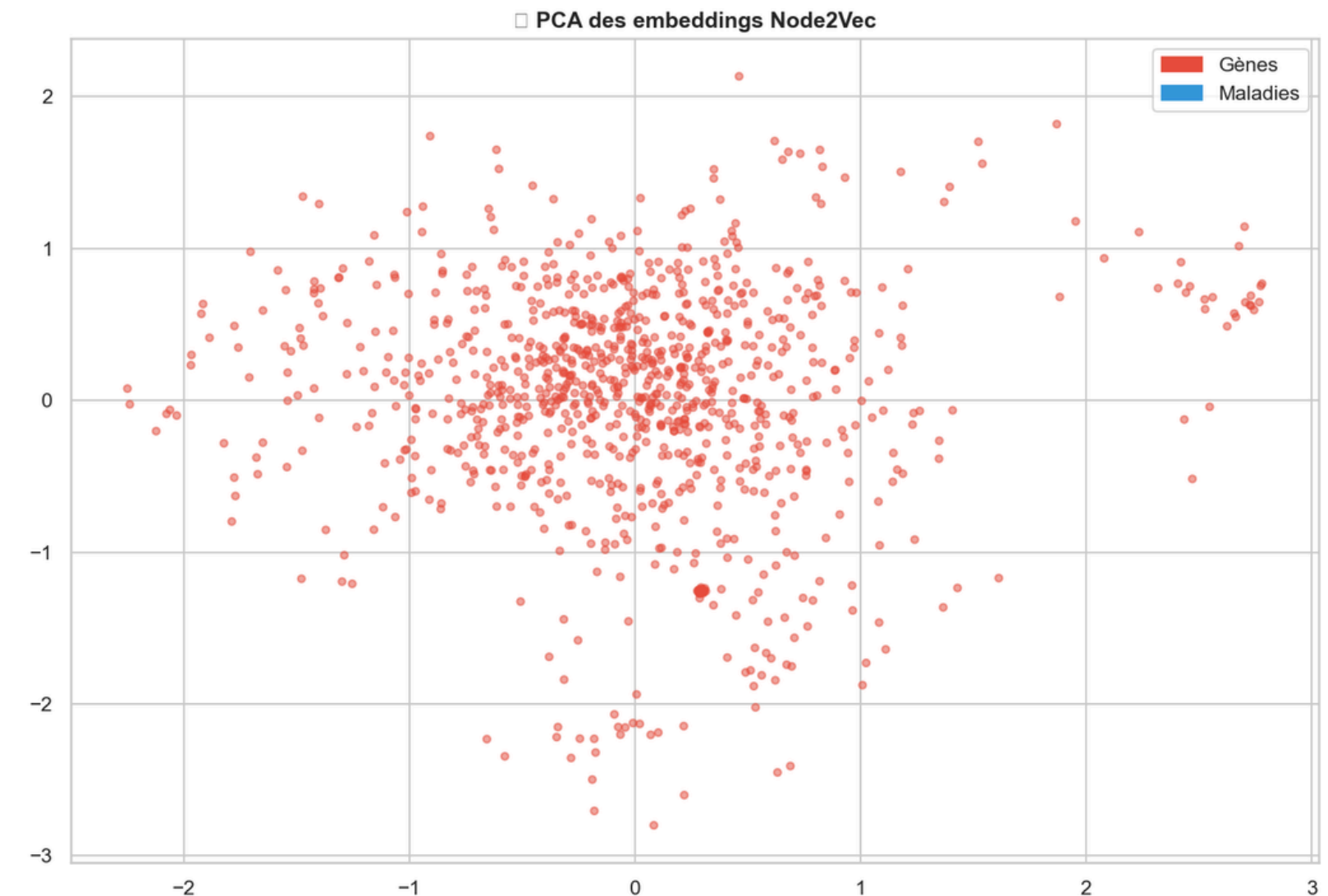
Analogie Word2Vec

Node2Vec transpose Word2Vec aux graphes :

- **Mots** → Nœuds du graphe
- **Phrases** → Chemins aléatoires
- **Embeddings** → Vecteurs 64D par nœud

Paramètres utilisés

Paramètre	Valeur	Description
dimensions	64	Taille du vecteur
walk_length	20	Longueur de chaque marche
num_walks	50	Marches par nœud
p (return)	1	Probabilité de retour
q (explore)	1	Probabilité d'exploration



Processus d'entraînement

1. Génération de marches aléatoires biaisées depuis chaque nœud
2. Entraînement Skip-gram sur les séquences de nœuds
3. Obtention d'un embedding de 64D pour chaque nœud
4. Combinaison Hadamard pour prédire les liens

Node2Vec – Résultats et Analyse de l'Échec

Résultat : AUC = 0.34-0.40 – performance inférieure au hasard (0.5)

Pourquoi ça a échoué ?

Graphe trop fragmenté

92% des nœuds ont un degré ≤ 1 dans G_{train} . Le graphe est quasi-déconnecté.

Marches bloquées

Marches bloquées sur des nœuds isolés, produisant des séquences non informatives.

PCA variance = 6.9%

Les embeddings capturent très peu de variance – ils sont quasi-aléatoires et non informatifs.

Ce que ça nous apprend

Cet échec n'est **pas une erreur** – c'est un résultat scientifique valable : le graphe est **fortement fragmenté**.

Les méthodes par marches aléatoires nécessitent une connectivité suffisante. Sur un graphe biparti creux, elles sont **inadéquates**.

Conséquence positive

Ce résultat justifie l'exploration d'approches **globales** comme la factorisation matricielle (GAE) qui ne dépendent pas de la connectivité locale.

GAE — Principe et Implémentation

Graph Autoencoder (GAE)

Le GAE utilise une **décomposition SVD** de la matrice d'adjacence :

Chaque nœud est représenté par un vecteur latent de 64 dimensions.

Pourquoi plus robuste ?

- **Opération globale** : décompose la matrice entière, indépendante de la connectivité locale
- **Sans marches** : fonctionne sur des graphes fragmentés
- **Variance** = 9.9% (vs 6.9% N2V)

Combinaison des features

Pour chaque paire (gène, maladie), on combine leurs embeddings par **produit de Hadamard** (élément par élément) : . Ce vecteur de 64 dimensions sert d'entrée au classifieur.

Paramètres du GAE

- **64 composantes** principales conservées
- Variance expliquée : **9.9%**
- Matrice d'adjacence : $16\,555 \times 16\,555$

Classifieurs testés

Régression Logistique

Classifieur linéaire simple

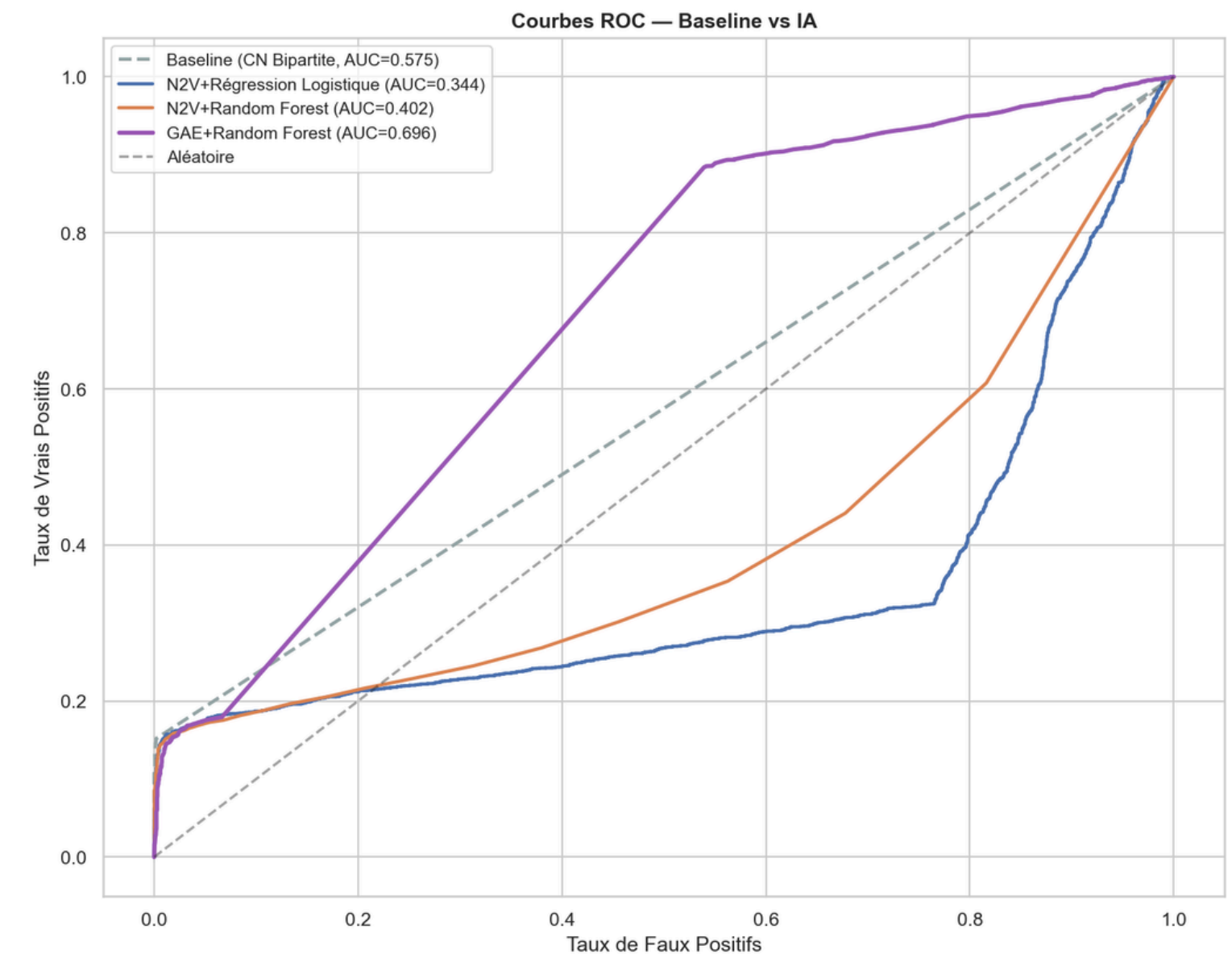
Random Forest

Ensemble d'arbres de décision

GAE — Résultats et Comparaison Finale

Méthode	AUC-ROC
GAE + Random Forest	0.6956 ★
GAE + Logistic Reg.	0.5608
N2V + Random Forest	0.4025
N2V + Logistic Reg.	0.3436
Baseline Jaccard	0.5751
Baseline CN	0.5751
Baseline PA	0.3704

Gain total : +12.05 points AUC
GAE+RF (0.6956) vs meilleure baseline (0.5751)



Conclusion : le GAE par factorisation SVD surpasse significativement toutes les autres méthodes. L'approche globale est mieux adaptée aux graphes biomédicaux creux et fragmentés que les approches locales (baselines) et les approches par échantillonnage (Node2Vec).

Système de Recommandation Final

Architecture du système



Niveaux de confiance



Réentraînement final

Le modèle final est réentraîné sur **100% des données** (train + test) pour les recommandations finales.

Classe GeneDiseaseRecommender avec deux fonctions :

➤ `recommend_genes_for_disease(disease_id, top_k)`

➤ `recommend_diseases_for_gene(gene_id, top_k)`

Exemple d'utilisation

```
recommender = GeneDiseaseRecommender(model, embeddings, G_full)
```

Top-5 gènes pour une maladie

```
recs = recommender.recommend_genes_for_disease("D_ORPHA:528084",  
top_k=5)
```

Top-5 maladies pour un gène

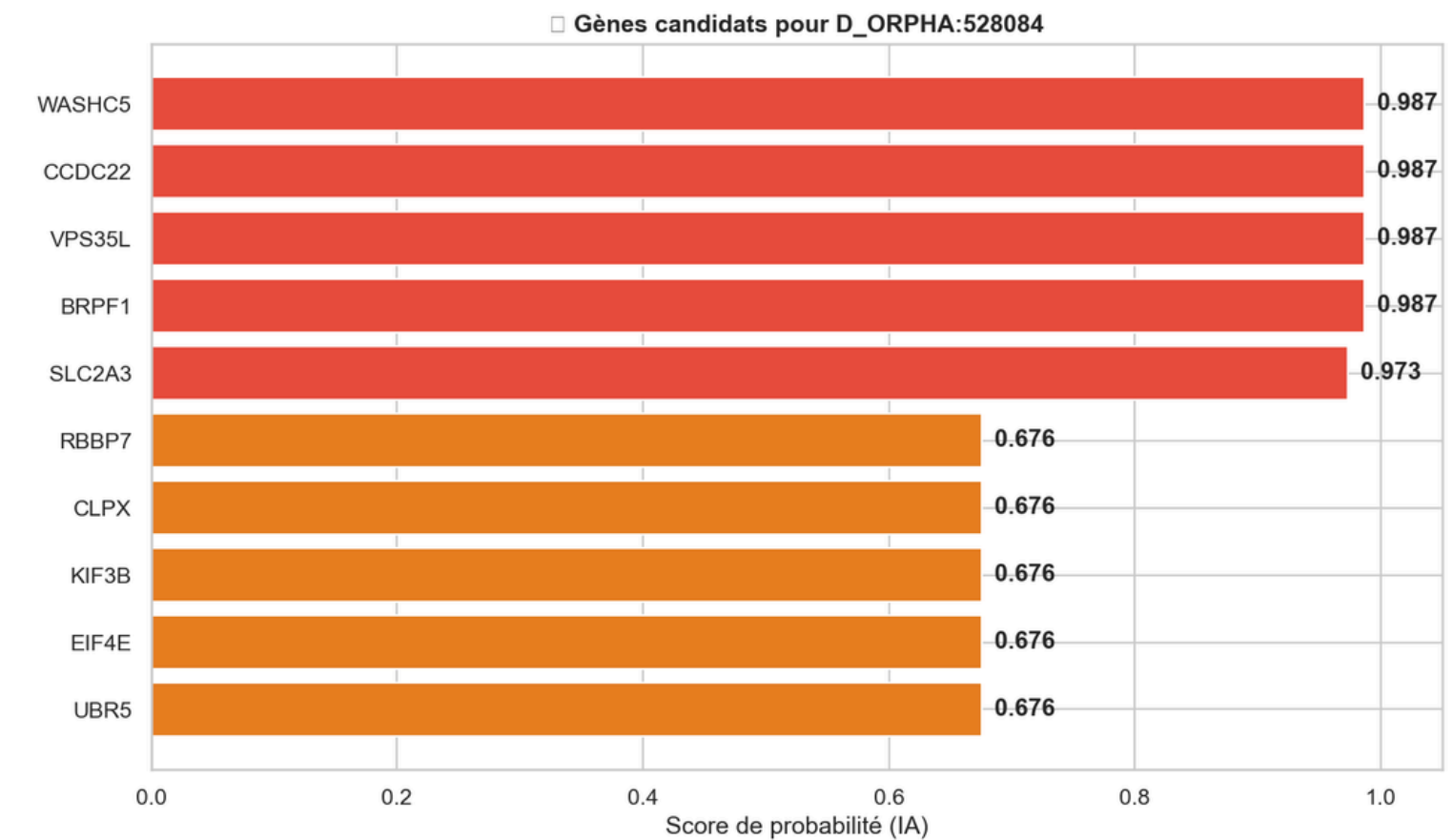
```
recs = recommender.recommend_diseases_for_gene("G_3043",  
top_k=5) # HBB
```


Démonstrations Concrètes

Exemple 1 : Maladie → Top gènes candidats

D_ORPHA:528084 (114 gènes déjà connus dans HPO)

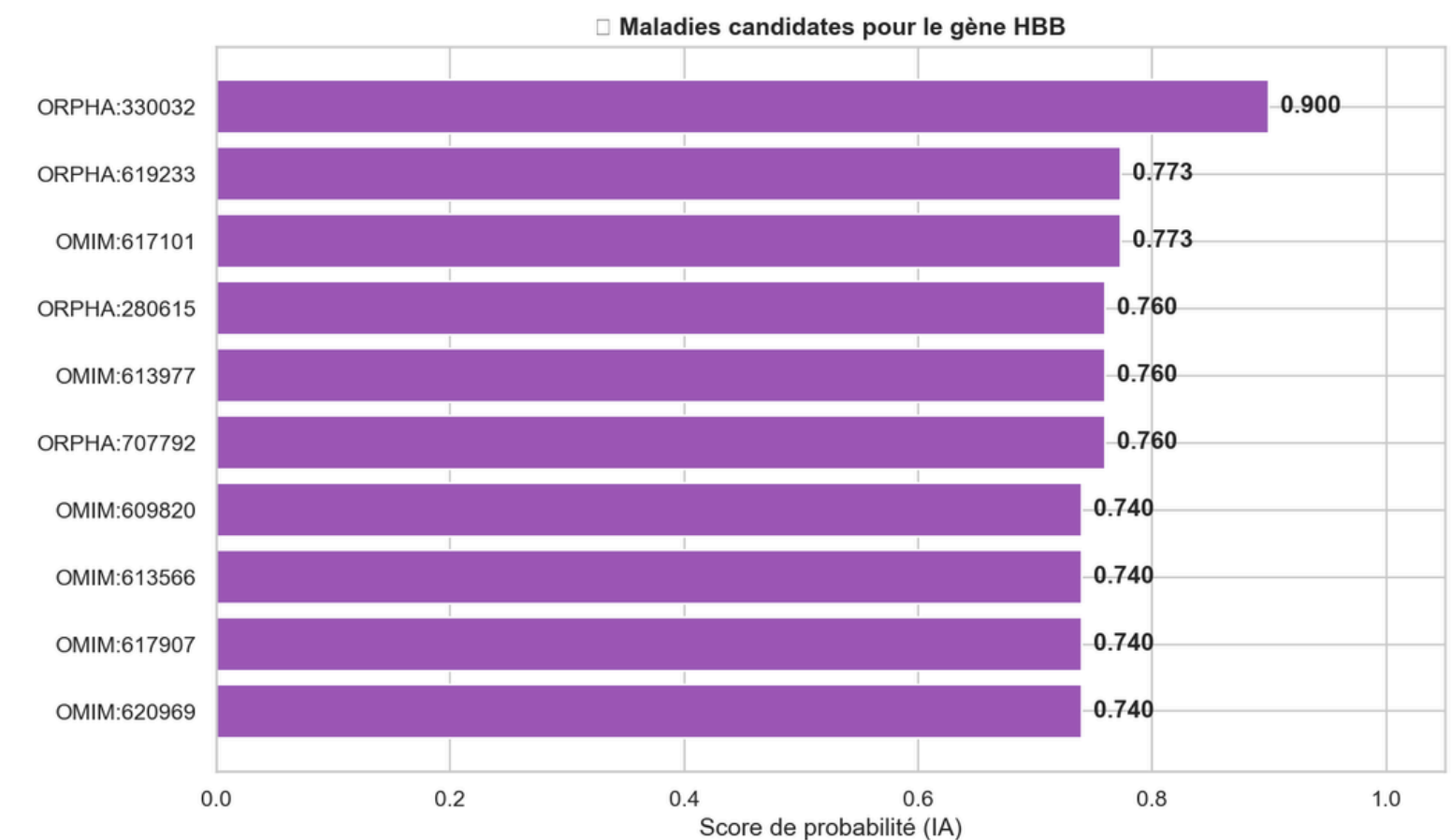
Rang	Gène	Score IA	Confiance
1	WASHC5	0.987	Très Haute
2	CCDC22	0.987	Très Haute
3	VPS35L	0.987	Très Haute
4	BRPF1	0.987	Très Haute
5	SLC2A3	0.973	Très Haute



Exemple 2 : Gène → Top maladies candidates

HBB – Gène de l'hémoglobine bêta (37 maladies connues)

Rang	Maladie	Score IA	Confiance
1	ORPHA:330032	0.900	Très Haute
2	ORPHA:619233	0.773	Très Haute
3	OMIM:617101	0.773	Très Haute
4	ORPHA:280615	0.760	Très Haute
5	OMIM:613977	0.760	Très Haute



Limites et Perspectives d'Amélioration

1 Échec de Node2Vec

Documenté par la forte fragmentation. Marches aléatoires inefficaces sur des graphes avec 92% de nœuds de degré ≤ 1 .

2 Biais vers les nœuds de faible degré

Cohérent avec PA AUC=0.37. Le modèle apprend un raccourci statistique : les nœuds rares ont plus de chances d'être de vrais liens.

3 Qualité des exemples négatifs

Certains "négatifs" pourraient être de vrais liens inconnus. Le bruit dans les labels limite le plafond de performance atteignable.

4 Graphe unimodal

Seules les interactions gène-maladie sont utilisées. L'ajout d'autres modalités (protéines, pathways) pourrait enrichir les prédictions.

Conclusion

Les méthodes de Graph Mining appliquées à l'intelligence artificielle constituent une approche prometteuse pour accélérer la découverte de nouvelles associations gène-maladie.

Merci pour votre attention